

MODULE 08 · DOMAIN 4 - TESTS, DOCS AND REVIEW

Leaving It Better

Tests, monitors, and the documentation nobody has been able to afford until now.

DOMAIN 4 of 5 MODULE 8 of 11 PROMPTS 3

What you know that nobody wrote down

By this point in a piece of work, you are holding a lot

- ◆ The grain, stated as a sentence, because you made it state one.
- ◆ Every place the document and the code disagree.
- ◆ Assumptions the query silently depends on. Nulls that must not appear.
- ◆ Which movement codes, which filters, which decisions nobody recorded.

THE PROBLEM

All of that currently lives in your head and in a chat window you will close this afternoon.

The grain sentence is a test suite that has not been written yet.

Four tests fall out of one sentence

One row per part per site

- ◆ **Uniqueness.** Part and site together must have no duplicates.
- ◆ **Referential.** Every key resolves. No rows parked on the default value.
- ◆ **Reconciliation.** The measure total ties back to the source.
- ◆ **Null and range.** Nullable columns behave, measures stay in plausible bounds.

THE POINT

You did not need to invent any of these. They are implied by the grain and the join path.

What you actually type

Give it the grain and the join path. It has everything it needs.

Grain: one row per part per site.
Join path and target DDL are below.

Write the checks that would fail if this table were wrong:

1. Uniqueness on the grain key
2. Every dimension key resolves, none on the default
3. Measures reconcile to the source aggregate
4. Nullable columns behave as specified

Each check returns zero rows when healthy. No thresholds.

For each, name the defect it catches and one it would miss.

THE CONVENTION THAT HELPS

Zero rows when healthy makes the whole suite readable at a glance.

Zero rows means healthy

Four checks. Any output at all is a failure.

```
01 -- 1. Grain uniqueness
02 SELECT part_no, site, COUNT(*) FROM fct_usage
03 GROUP BY part_no, site HAVING COUNT(*) > 1;
04
05 -- 2. Unresolved dimension keys
06 SELECT COUNT(*) FROM fct_usage WHERE part_key = -2
07 HAVING COUNT(*) > 0;
08
09 -- 3. Reconciliation to source
10 SELECT ABS(f.total - s.total) FROM (...) f, (...) s
11 WHERE ABS(f.total - s.total) > 0.01;
```

THE ONE THAT PAYS

Check two is the one that would have caught the null hash keys from the migration module.

A test and a monitor are not the same thing.

Two different jobs

A TEST

Runs when code changes

- Known data, known answer
- Binary pass or fail
- Blocks a deployment
- Proves the code is right

A MONITOR

Runs when data arrives

- New data, unknown answer
- Threshold and trend
- Raises an alert
- Proves the assumption still holds

THE TIEBREAKER

Everything in this course that broke silently would have been caught by a monitor, not a test.

What you actually type

Feed it the assumptions you discovered, not the code.

Below are assumptions this pipeline depends on, discovered during development. None is enforced anywhere.

- site_code is never null on the source table
- part numbers and contract ids never collide
- the movement type list has not changed

For each, write a monitor that runs after every load. Give me:

- The query, returning a count
- A threshold, and your reasoning for that number
- What a breach actually means, in business terms

If an assumption cannot be monitored, say so and explain why.

THE INPUT THAT MATTERS

Feeding it assumptions rather than code is what makes this work. It cannot infer them.

The assumption from module seven

Zero today. That is exactly why it needs watching.

```
01 -- Assumption: site_code is never null
02 -- Consequence if breached: hash key becomes null,
03 --   row fails every downstream join, lands on default key
04
05 SELECT COUNT(*) AS null_site_codes
06 FROM   src_part_site
07 WHERE  site_code IS NULL;
08
09 -- Threshold: 0. Not a tolerance. Any row breaks a key.
10 -- Current value: 0. Has been 0 for four years.
```

WHY THIS ONE

An assumption that has held for four years is the one most likely to be undefended.

Now regenerate the document. From the code that actually runs.

Why this is possible now and was not before

- ◆ **It was never a hard problem.** It was an expensive one.
- ◆ Documenting eight hundred lines by hand took a week nobody would fund.
- ◆ So it happened once, at go live, and never again.
- ◆ That is the entire mechanism behind every drift finding in this course.

WHAT CHANGED

The economics changed. A regenerated mapping document is now an afternoon, not a week.

What you actually type

Describe behaviour. Refuse to guess at purpose.

```
Generate a mapping document for the target table below, from  
the deployed procedure. Column layout: <paste header>
```

```
For each target column give the source table, source column,  
and the transformation exactly as the code performs it.
```

- Describe what the code does, never what it is for
- Quote literal values in full. Never summarise a code list

```
Then a section: BEHAVIOUR WITHOUT STATED REASON. Anything the  
code does with no explanation available in the code.
```

```
Mark the document as generated from code, with today's date.
```

THE RULE THAT MATTERS

Quote literals in full, or sixteen codes become 'a list' again.

What the regenerated document is, and is not

- ◆ **It is accurate.** It describes the code that runs tonight, because it was read from it.
- ◆ **It is complete.** Every column, every literal, every filter.
- ◆ **It has no intent.** It cannot say why any rule exists.
- ◆ **It is not a specification.** Nobody agreed to it. It is a description.

KEEP BOTH

Two artifacts, two jobs. The original says what was wanted. This one says what happens.

You generate the four grain tests. The uniqueness check fails immediately, returning 340 duplicate part and site pairs on a table that has been in production for three years.

The reconciliation check passes. Totals tie to source exactly.

How do you read this?

- ☐ A The grain sentence is wrong. The real grain is finer
- ☐ B There is a fan-out and the totals are inflated
- ☐ C The uniqueness check is wrong for this table
- ☐ D The reconciliation check is too loose to detect it

◆ PAUSE AND DECIDE

How do you read this?

- A

 The grain is finer
The likely answer. Duplicates plus correct totals means a real dimension you did not name.
- B

 Fan-out, inflated totals
A fan-out would inflate the measures. Reconciliation passed, so the totals are not inflated.
- C

 Bad check
The check tests exactly what the grain sentence claims. If it fails, the sentence is the suspect.
- D

 Loose reconciliation
It reconciled to a penny. That is not a loose check by any reading.

THE SIGNAL

Duplicates with correct totals is the signature of a missing grain column, not a defect.

You propose a monitor with a threshold of zero on null site codes. A colleague objects that a hard zero will page somebody the first time a single bad row arrives.

They suggest a threshold of ten to reduce noise.

What is the right response?

- ☐ A Agree. Ten is a reasonable noise filter
- ☐ B Hold at zero. One row breaks a key
- ☐ C Keep zero, but route it to a queue rather than a page
- ☐ D Set it to zero and review after a month

◆ PAUSE AND DECIDE

What is the right response?

- A

 Threshold of ten
Ten rows with null keys is ten rows silently on a default. The threshold hides the thing being watched.
- B

 Hold at zero
Correct on the number and it loses the argument. Their objection about paging is legitimate.
- C

 Zero, but not a page
Keeps detection exact and fixes the real complaint, which was severity, not sensitivity.
- D

 Review in a month
Defers a decision you can make now. The consequence of a null key is already known.

THE SIGNAL

Their objection was about severity. Answering it with sensitivity gives away the detection.

The regenerated mapping document is accurate and complete. Your manager suggests replacing the original document with it, since the original is two years out of date.

The original contains business context that the code does not.

What do you recommend?

- ☐ A Replace it. Accurate beats out of date
- ☐ B Keep both, clearly labelled by what each one is
- ☐ C Merge them into a single corrected document
- ☐ D Keep the original and attach the generated one as an appendix

◆ PAUSE AND DECIDE

What do you recommend?

- A

 Replace it
Destroys the only record of intent you have, to fix a problem about behaviour.
- B

 Keep both, labelled
They answer different questions. Labelling is what stops the generated one being read as agreed.
- C

 Merge them
Produces one document where you can no longer tell which parts anyone agreed to.
- D

 Generated as appendix
Close, and it subordinates the accurate document to the stale one. Wrong way round.

THE SIGNAL

The moment you cannot tell description from specification, you have lost both.

The BEHAVIOUR WITHOUT STATED REASON section of the generated document lists nine items. Seven are trivial. Two are substantive filters excluding whole categories of data.

You are at the end of the sprint.

What do you do with the two substantive items?

- ☐ A Leave them in the document. It is recorded, which is the point
- ☐ B Raise them as tickets to investigate later
- ☐ C Ask the two longest-serving people on the team, today
- ☐ D Remove the filters, since nobody can justify them

◆ PAUSE AND DECIDE

What do you do with the two substantive items?

A Recorded is enough

Better than nothing and it preserves the mystery rather than resolving it.

C Ask people today

Institutional memory walks out of the building. It is free right now and gone in a year.

B Tickets for later

The honest version of A. Nobody has ever picked up the ticket that says find out why this filter exists.

D Remove the filters

Unexplained is not unjustified. That distinction is the whole course.

THE SIGNAL

The document made the questions askable. Asking them is a ten minute conversation with a deadline.

Take this to work tomorrow

- ◆ Turn your grain sentence into four checks. It takes one prompt.
- ◆ Write down every assumption you found, then generate a monitor for each.
- ◆ Regenerate the mapping document for one table. Mark it as generated, with the date.
- ◆ Whatever comes back unexplained, ask a human today rather than filing it.

THE PAYOFF

None of this is a project. It is one afternoon at the end of work you have already done.

Drift is not getting solved. It is getting
cheap to correct.